SENG401 – GRADUATION PROJECT

Software Design Document

By Sinem Coşkun, Al-baraa Abdullah Abdullah Abdullah Al-yadumi

TABLE OF CONTENTS

1. Introduction....5
1.1 Purpose of the SDD....5
1.2 Scope....5
1.3 Definitions, Acronyms, and Abbreviations....5
1.4 Document Overview....6
2. System Overview....7
2.1 High-Level System Description....7
2.2 Major System Components....7
2.3 Design Constraints and Assumptions....9
2.4 High-Level Architecture Diagram....10
3. Architectural Design....12
3.1 Architectural Style and Layers....12
3.2 Subsystem Decomposition....12
3.2.1 Attacker Layer (External Actor)....12
3.2.2 Honeypot Layer....13
3.2.3 Data Transfer and Storage Layer....13
3.2.4 LLM Deception Pipeline....14
3.2.6 Data Export & Research Layer....15
3.3 Component Interaction Overview....15
3.4 Architectural Diagrams:....17
4. Data Design....20
4.1 Data Flow Overview....20
4.2 Logical Data Model — SQLite....21
4.2.1 Raw Log Tables....21
4.2.2 LLM_Log Table....21
4.3 Logical Data Model — Offline Parquet Datasets....21
4.3.1 Session Intelligence Dataset (session_intelligence.parquet)....21
4.3.2 ML Evaluation Metrics (model_results.json)....22
4.4 Data Flow Diagram (DFD)....22
4.5 Data Dictionary (High-Level)....23

5. Component Design....24
5.1 Honeypot Layer Components....25
5.1.1 Cowrie Component (SSH Honeypot)....25
5.1.2 Honeytrap Component (HTTP/FTP Emulation)....25
5.1.3 Suricata Component (Network IDS)....25
5.2 LLM Deception Pipeline Components (Integration Layer)....26
5.2.1 LLM Gateway Component (Phi-3)....26
5.2.2 Output Filter Component....26
5.3 Data Transfer and Storage Components....27
5.3.1 Log Normalization & Aggregation Component (Edge)....27
5.3.2 MQTT Publisher Component (Edge Device)....27
5.3.3 MQTT Broker Component....28
5.3.4 SQLite Storage Component....28
5.4 Offline Analytical Intelligence Components....29
5.4.1 Session Construction Component....29
5.4.2 Feature Extraction & Data Leakage Check Component....29
5.4.3 ML Classification & Anomaly Detection Component....30
5.5 Data Export and Research Components....30
5.5.1 Parquet / JSON Exporter Component....30
5.5.2 Offline Analysis Environment (Jupyter Notebooks)....31
6. Algorithm Design....31
6.1 Offline Session Construction Algorithm....31
6.2 Data Leakage Prevention & Feature Selection Algorithm....31
6.3 Machine Learning Classification Workflow....32
6.4 LLM Command Routing Algorithm (Deception Pipeline)....33
6.5 Algorithm Dependencies....33
7. User Interface Design....33
7.1 Interface Overview....34
7.2 Analytical Interface (Jupyter & Pandas)....34
7.3 Deceptive Terminal Interface (LLM-Powered)....34
7.4 Interaction Scenarios....35

8. Non-Functional Design 35
8.1 Performance Design 35
8.2 Reliability Design 36
8.3 Security Design 36
8.4 Usability Design 37
8.5 Maintainability Design 37
8.6 Extensibility Design 38
8.7 Portability Design 38
9. Deployment & Runtime View 38
9.1 Physical Deployment Diagram 39
9.2 Runtime Scenarios (Sequence Diagrams) 40
9.2.1 Runtime Scenario 1 — LLM Deception & Edge Logging (Real-Time) 40
9.2.2 Runtime Scenario 2 — Offline ML Intelligence Pipeline (Batch Processing) 41
10. SRS Traceability 42
10.1 Functional Requirements Traceability 42
10.2 Non-Functional Requirements Traceability 45

1. Introduction

1.1 Purpose of the SDD

The purpose of this Software Design Document (SDD) is to provide a detailed technical description of the system design for the "Detecting Protocol Tunneling Activities and Cyber Attacks with Machine Learning-Based IoT Honeypot." project.
This document translates the functional and non-functional requirements defined in the SRS into a comprehensive design plan that guides implementation, integration, testing, and deployment. The SDD ensures consistent and sustainable configuration of all components, including the honeypot layer, log aggregation, offline session building, leakage-aware feature extraction, machine learning classification, and the LLM-powered deception pipeline.

1.2 Scope

This SDD covers the entire design of the IoT honeypot-based detection system described in the SRS. It includes architectural design, subsystem descriptions, data models, algorithms, interaction flows, sequencing diagrams, deployment views, and mappings to SRS requirements.

Here's what the design covers:

- It runs an IoT honeypot right on the edge device.
- Logs get collected and sent out using MQTT.
- All those logs end up in a central SQLite database.
- **(Offline Intelligence)** Before ML analysis, it constructs temporally coherent behavioral sessions and extracts 52 traffic features.
- **(Data Leakage Prevention)** It applies strict automated leakage checks to separate SAFE features from HEURISTIC features to ensure models learn genuine behavior.
- **(ML Classification)** Provides Machine Learning classification across 11 multi-label attack categories, tunneling detection, and bot vs. human profiling.
- **(LLM Deception)** Enhances the Cowrie honeypot with a fine-tuned LLM Gateway (Phi-3) to dynamically generate realistic synthetic terminal responses for commands.
- **(Data Export)** Exports comprehensive session intelligence as .parquet and .json files for offline research instead of relying on real-time dashboards.

This document only reflects capabilities defined in the SRS and does not introduce new functionalities.

1.3 Definitions, Acronyms, and Abbreviations

- **IoT:** Internet of Things
- **DMZ:** Demilitarized Zone (isolated network for honeypots)
- **Honeypot:** A fake system built to lure hackers and record what they do

- **MQTT:** Message Queuing Telemetry Transport
- **SQLite:** Lightweight relational database for log/event storage
- **Data Leakage:** The unintentional inclusion of heuristic features in ML training, which is prevented in this system to ensure scientifically valid models.
- **DNS:** Domain Name System
- **HTTP:** Hypertext Transfer Protocol
- **ML:** Machine Learning
- **Entropy:** A way to measure how random data is, which helps spot tunneling
- **Feature Extraction:** Turning raw log data into useful numbers for analysis
- **Brute Force Attack:** Trying lots of passwords over and over to break in
- **Port Scan:** Systematic scanning of open ports
- **Malware Download:** Trying to download or install malicious files
- **Bot vs Human Behavior:** Figuring out if actions come from a person or a bot
- **LLM (Large Language Model):** An AI model (specifically Phi-3-mini-4k-instruct via QLoRA) used to generate synthetic, deceptive terminal responses.
- **Offline Session Analysis:** Aggregating raw logs into behavioral sessions (e.g., using a 30-minute inactivity threshold) for batch processing instead of real-time packet inspection.
- **SAFE / HEURISTIC Features:** Features extracted from logs; SAFE features are used for ML, while HEURISTIC features are excluded to prevent data leakage.
- **Macro-F1 Score:** The primary ML evaluation metric used to handle extreme class imbalances.
- **Parquet:** A columnar storage file format used to export the final intelligence datasets.

1.4 Document Overview

The remainder of this SDD is structured as follows:

- **Section 2:** High-level system description and architecture overview.
- **Section 3:** Architectural design, subsystem decomposition, component interactions, and diagrams.
- **Section 4:** Data design, logical models, and data flow diagrams.
- **Section 5:** Component-level design for each subsystem.
- **Section 6:** Algorithms for offline session construction, leakage-aware feature selection, ML classification, and LLM command routing.
- **Section 7:** Analytical interfaces and offline dataset export design.
- **Section 8:** Non-functional design considerations (including LLM latency and security).
- **Section 9:** Deployment model and runtime sequence diagrams.
- **Section 10:** Traceability between SRS requirements and SDD design elements.

2. System Overview

2.1 High-Level System Description

The system is a comprehensive IoT honeypot-based behavioral intelligence and deception platform designed to capture, structure, and analyze cyber attacks using advanced machine learning and Large Language Models (LLMs)..
A Raspberry Pi device acts as the edge honeypot, exposed to the Internet through a DMZ/guest network, allowing it to receive real-world malicious traffic.
Three open-source security tools—Cowrie, Honeytrap, and Suricata—simulate vulnerable services and capture attacker interactions, commands, sessions, and packet-level activity.

To increase honeypot realism, the Cowrie honeypot is enhanced with an LLM-Powered Deception Pipeline (using a fine-tuned Phi-3 model) that generates dynamic, context-aware synthetic Linux terminal responses for unrecognized commands.

All collected logs are transferred to a central analysis machine through the lightweight MQTT protocol and stored in a SQLite database.

The data processing relies on a strict **Offline Analytical Intelligence Pipeline:**

1. **Offline Session Construction:** Raw events are temporally grouped into coherent attacker sessions based on inactivity thresholds.
2. **Leakage-Aware Feature Extraction:** 52 traffic features (such as entropy, URI length, inter-command delays) are extracted. The system rigorously applies a Data Leakage Check to separate SAFE features (used for ML) from HEURISTIC features (excluded from ML) to ensure models learn genuine behavior.
3. **Machine-Learning Classification:** Utilizing tree-based models (XGBoost, Random Forest) and Anomaly Detection (Isolation Forest) to identify brute-force attempts, tunneling, malware droppers, and classify sessions across 11 multi-label attack categories and bot-vs-human behavior.

2.2 Major System Components

1. Attacker (External Actor)

- Source of real-world cyber attacks
- Initiates SSH, HTTP/FTP, DNS and other network traffic

2. Honeypot Layer

Components:

- Cowrie
- Honeytrap
- Suricata

Responsible for:

- Capturing commands, sessions, requests, network alerts
- Generating time-stamped raw logs

3. Data Transfer and Storage Layer

Components:

- Log Aggregator (Raspberry Pi)
- MQTT Publisher (Raspberry Pi)
- MQTT Broker (Central Machine)
- SQLite Database

Responsible for:

- Standardizing logs
- Publishing logs to MQTT
- Receiving and storing logs in SQLite

4. LLM Deception Pipeline (Integration Layer)

Components:

- LLM Gateway (Phi-3-mini-4k-instruct via LoRA adapter)
- Output Filter

Responsible for:

- Receiving unknown SSH commands
- Generating synthetic Debian terminal outputs
- Sanitizing responses to prevent prompt injection.

5. Offline Analytical Intelligence Layer

Components:

- Session Builder
- Feature Extractor
- Data Leakage Checker
- ML Classification Module (XGBoost/RandomForest).

Responsible for:

- Grouping events into sessions
- Computing 52 features
- Explicitly excluding 28 leaked/heuristic features
- Training classifiers
- Assigning 11 multi-label attack categories

6. Data Export & Research Layer

Components:

- Parquet/JSON Exporter
- Offline Analysis Environments (Jupyter Notebooks, Pandas).

Responsible for:

- Storing analytical intelligence and providing interfaces for scientific evaluation of Macro-F1, PR-AUC, and ROC-AUC metrics

2.3 Design Constraints and Assumptions

The system is designed under the following constraints derived from the SRS:

Hardware Constraints

- Raspberry Pi 4 Model B (limited CPU, RAM, and storage)
- Must run honeypots continuously while exposed to the Internet
- LLM Fine-Tuning Constraint: Requires access to a GPU environment (e.g., Google Colab T4) to train the QLoRA adapter

Network Constraints

- Raspberry Pi must be placed in DMZ / Guest Network
- No connection to production or university networks

Software and Tooling Constraints

- Only open-source tools may be used:
    - Cowrie, Honeytrap, Suricata
    - MQTT

    - SQLite
    - Python (Pandas, Scikit-learn, XGBoost)
    - HuggingFace Ecosystem (PEFT, QLoRA)

Analytical Constraints

- Tunneling detection must reach:
    - **≥ 80% Macro-F1 score**
- ML classifiers must reach:
    - **≥ 80% Macro-F1 score**

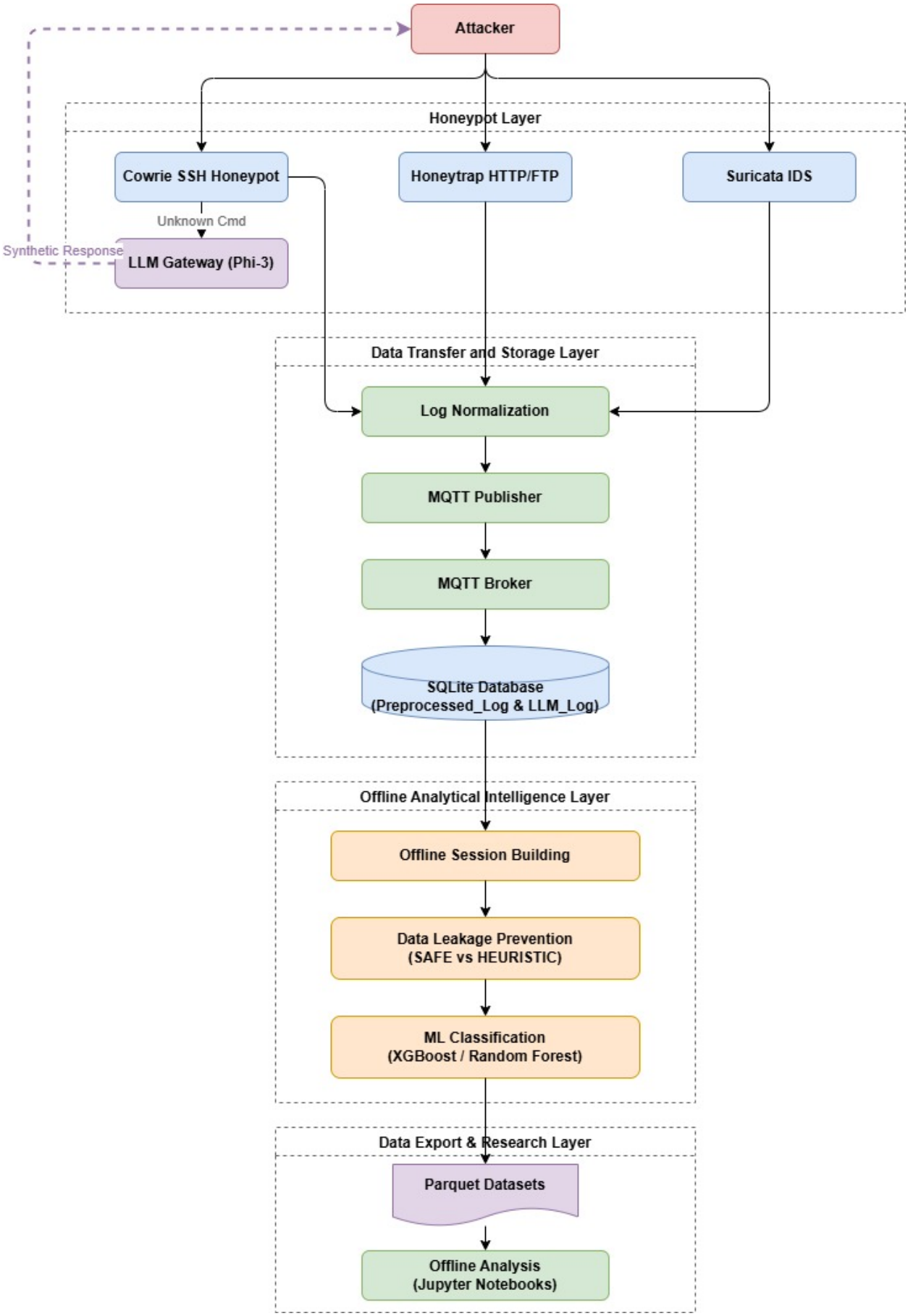
Operational Assumptions

- Raspberry Pi will receive sufficient real attack traffic
- MQTT communication remains stable
- SQLite can handle log volume
- Python scripts run on central machine
- Central analysis is performed offline in batch processes rather than continuous streaming.

2.4 High-Level Architecture Diagram

This subsection will contain the main system diagram describing:

- Attacker ↔ Honeypot Layer (Cowrie + LLM Gateway) → Log Normalization → MQTT → SQLite → Offline Session Building & Leakage Prevention → ML Classification → Parquet Datasets → Offline Analysis


Attacker
Honeypot Layer
Cowrie SSH Honeypot
Honeytrap HTTP/FTP
Suricata IDS
Unknown Cmd
Synthetic Response
LLM Gateway (Phi-3)
Data Transfer and Storage Layer
Log Normalization
MQTT Publisher
MQTT Broker
SQLite Database
(Preprocessed_Log & LLM_Log)
Offline Analytical Intelligence Layer
Offline Session Building
Data Leakage Prevention
(SAFE vs HEURISTIC)
ML Classification
(XGBoost / Random Forest)
Data Export & Research Layer
Parquet Datasets
Offline Analysis
(Jupyter Notebooks)

3. Architectural Design

In this section, the architectural structure of the system is explained using a layered model. The design reflects how data flows from external attackers to honeypot servers, through the LLM deception loop, into structured offline storage, and finally through the machine learning pipeline into research-ready datasets.

All architectural details are strictly derived from the SRS.

3.1 Architectural Style and Layers

The system follows a **layered architecture** where each layer has a separate responsibility in the overall data processing and analysis process.

The architectural layers are:

1. **Attacker Layer**
2. **Honeypot Layer**
3. **Data Transfer and Storage Layer**
4. **LLM Deception Pipeline**
5. **Offline Analytical Intelligence Layer**
6. **Data Export & Research Layer**

Each layer processes specific inputs and produces outputs that flow to the next layer.

The design ensures modularity, ease of maintenance, clear separation of responsibilities, and ease of future expansion.

3.2 Subsystem Decomposition

This section describes each architectural subsystem in detail.

3.2.1 Attacker Layer (External Actor)

Role:
It represents real-world attackers who generate malicious traffic targeting the Raspberry Pi device.

Responsibilities:

- Initiate SSH brute-force attempts
- Send HTTP/FTP requests
- Perform port scans
- Generate DNS and HTTP traffic used for tunneling detection
- Attempt malware-related actions

Outputs:

- Malicious traffic reaching the Honeypot Layer

3.2.2 Honeypot Layer

This layer simulates vulnerable services to capture attacker interactions.

Components:

- **Cowrie** — SSH/Telnet honeypot
- **Honeytrap** — HTTP/FTP service emulation
- **Suricata** — IDS for packet inspection

Responsibilities:

- Interact with attackers
- Log commands, sessions, HTTP requests, and IDS alerts
- Timestamp all events
- Generate protocol-specific log files

Outputs:

- Raw log entries (SSH, HTTP/FTP, network alerts)

Consumes:

- Incoming attacker traffic

3.2.3 Data Transfer and Storage Layer

This layer handles log aggregation, reliable transfer, and structured storage.

Components:

- Log Aggregator (Raspberry Pi)
- MQTT Publisher (Raspberry Pi)
- MQTT Broker (Central Machine)
- SQLite Database

Responsibilities:

- Collect logs from honeypots and standardize them into a unified 18-field schema.
- Forward logs securely using MQTT over TLS (Port 8883).

- Receive logs at the central server
- Parse and validate log entries
- Store logs and extracted features in SQLite

Outputs:

- Structured SQLite datasets ready for offline batch processing.

Consumes:

- Honeypot-generated logs and LLM interactions.

3.2.4 LLM Deception Pipeline

This new subsystem generates dynamic, realistic system responses to prevent honeypot fingerprinting.

Components:

- LLM Gateway (Fine-tuned Phi-3-mini-4k-instruct via QLoRA adapter)
- Output Filter (Sanitization Module)

Responsibilities:

- Receive unrecognized attacker commands from the Cowrie Command Router.
- Generate contextually accurate, synthetic Debian terminal outputs.
- Sanitize outputs to prevent prompt injection, hallucination, or disclosure of real system states.

Outputs:

- Safe, synthetic terminal responses sent back to the attacker.

Consumes:

- Unrecognized commands from the Honeypot Layer.

3.2.5 Offline Analytical Intelligence Layer

This subsystem replaces real-time streaming with a robust, leakage-aware batch processing and machine learning pipeline.

Components:

- Session Builder
- Feature Extractor & Data Leakage Checker
- ML Classification Module (XGBoost, Random Forest, Isolation Forest)

Responsibilities:

- Group raw events into temporally coherent behavioral sessions (e.g., using a 30-minute inactivity gap).
- Compute 52 raw traffic features and rigorously separate 24 SAFE features from 28 HEURISTIC features to prevent data leakage.
- Train and apply ML models to classify tunneling, bot vs. manual behavior, and 11 multi-label attack categories.
- Compute robust performance metrics (Macro-F1, PR-AUC, ROC-AUC) utilizing 5-Fold Stratified Cross-Validation.

Outputs:

- Trained classification models, session intelligence profiles, and performance metrics.

Consumes:

- SQLite-stored structured logs.

3.2.6 Data Export & Research Layer

This subsystem replaces traditional dashboards by providing deep-dive analytical datasets for researchers.

Components:

- Parquet / JSON Exporter
- Offline Analysis Environments (Jupyter Notebooks, Pandas DataFrames)

Responsibilities:

- Export the final, leakage-free session features and ML classification results as .parquet files.
- Facilitate detailed scientific analysis, feature importance extraction, and metric evaluation.

Outputs:

- Exportable, open-source ready intelligence datasets.

Consumes:

- Feature arrays and classification results from the ML layer.

3.3 Component Interaction Overview

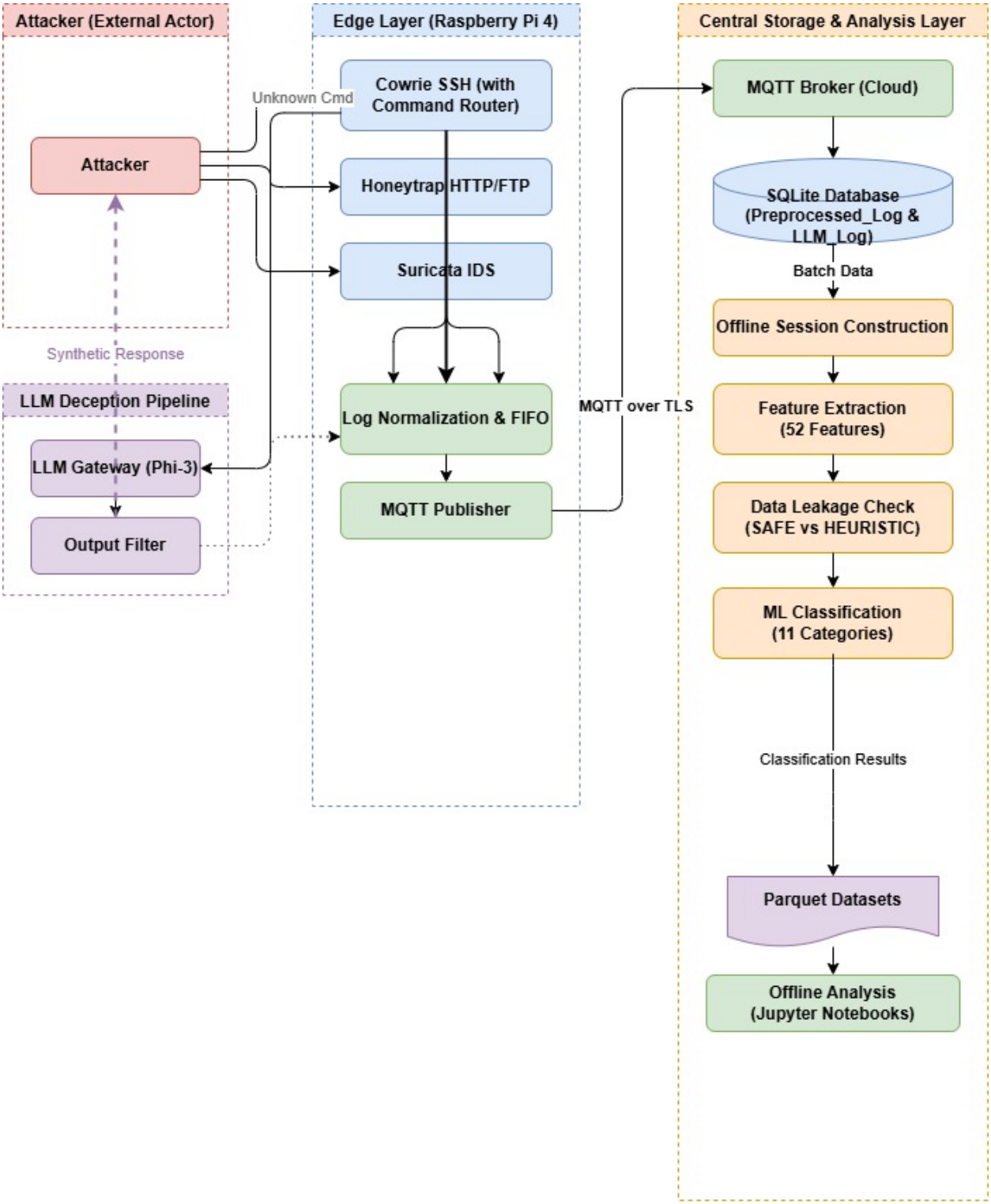
This section summarizes how the architectural subsystems interact:

1. **Attackers** generate malicious traffic toward the Raspberry Pi.
2. **Honeypot Layer** captures attacker interactions and generates logs.

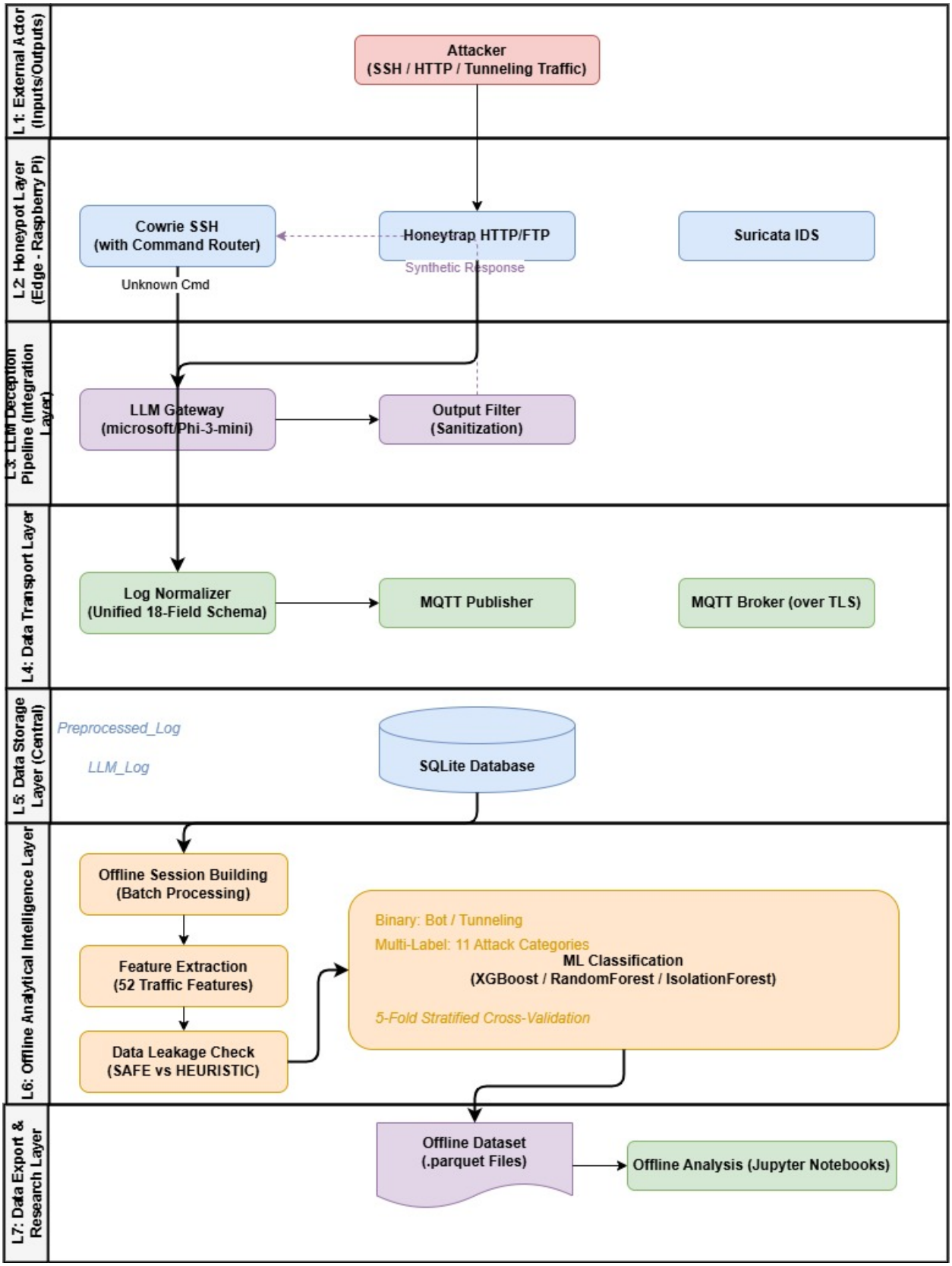
3. **The LLM Gateway** generates sanitized, synthetic terminal responses and returns them to the attacker to extend session duration.
4. **The Log Aggregator** standardizes all multi-protocol raw logs and LLM interactions into a unified 18-field schema via a FIFO pipe.
5. **MQTT Publisher** transmits logs to the central server.
6. **MQTT Broker** receives logs and forwards them.
7. **SQLite Storage** organizes logs for analysis.
8. **The Offline Session Builder** retrospectively groups isolated events into coherent attacker sessions.
9. **The Feature Extractor and Leakage Checker** compute 52 features, explicitly isolating 24 SAFE features to ensure ML models learn genuine behavior instead of deterministic rules.
10. **The ML Classification Module** utilizes tree-based models (e.g., XGBoost, Random Forest) to predict tunneling, bot behavior, and 11 attack types based strictly on the SAFE features.
11. Final behavioral intelligence and evaluation metrics (Macro-F1) are exported as .parquet datasets for offline Jupyter Notebook analysis.

3.4 Architectural Diagrams:

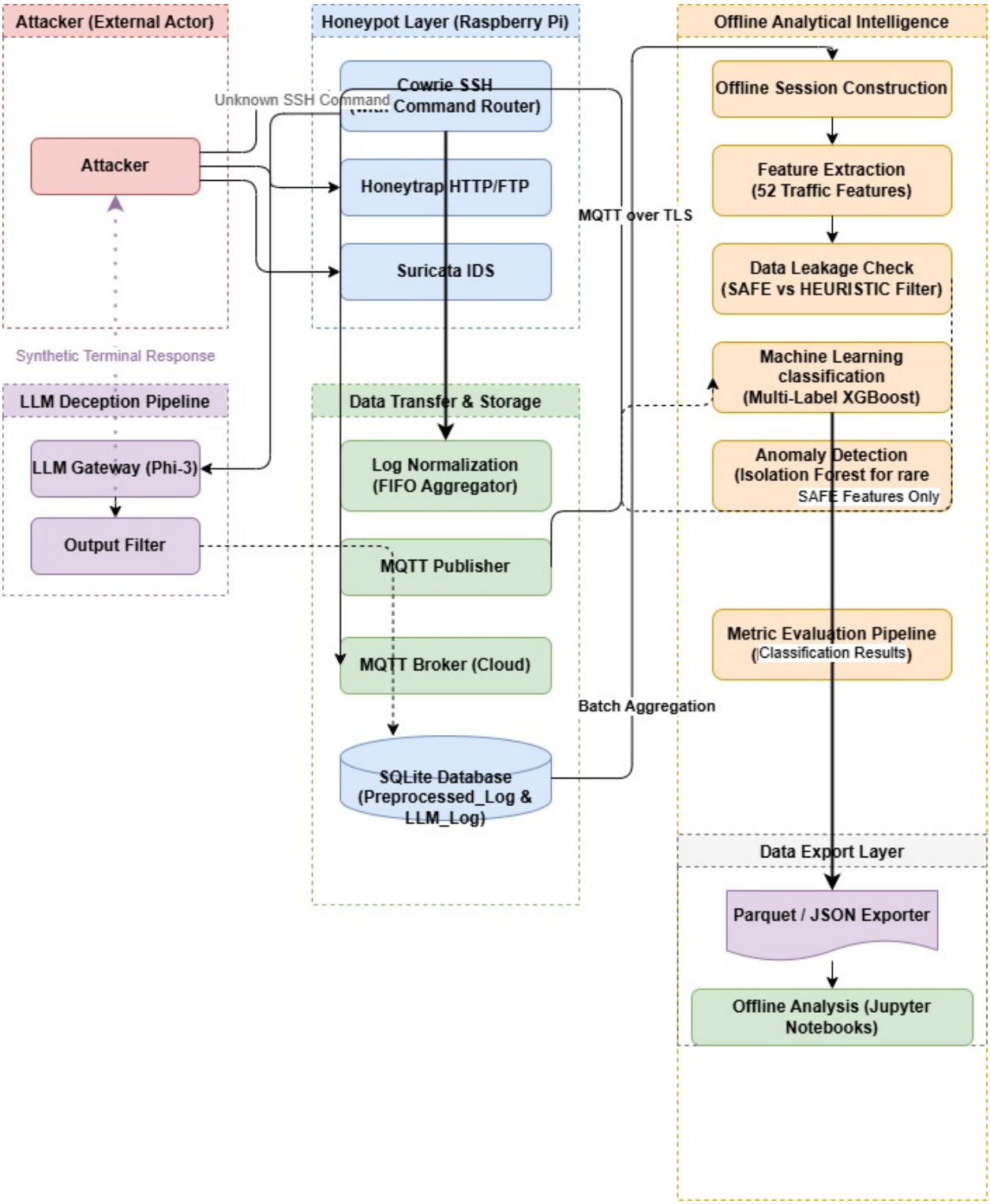
Component Interaction Diagram:



Layered Architecture Diagram:



Subsystem Diagram:



4. Data Design

This section describes how data is collected, transmitted, stored, transformed, and utilized across the system. All structures directly follow the SRS and reflect the offline analytical intelligence and LLM integration paradigms.

4.1 Data Flow Overview

The overall data lifecycle in the system follows these stages:

1. **Attackers generate malicious traffic**
   (SSH, HTTP/FTP, DNS, port scans, malware-related traffic)
2. **Honeypot Layer records raw logs**
   - Cowrie SSH commands, session details
   - Honeytrap HTTP/FTP interactions
   - Suricata IDS alerts and packet metadata
3. **Logs are aggregated and prepared on the Raspberry Pi**
   - Raw events from all three sources are instantly normalized into a unified 18-field schema and pushed to a FIFO pipe.
4. **MQTT transfers logs to the central server**
5. **SQLite stores structured log records**
   - Raw logs
   - Preprocessed logs and Extracted features
   - AI-Generated responses
6. **Offline Session Construction**
   - Raw events are grouped by (src_ip, dest_ip) with a 30-minute inactivity threshold to build temporally coherent sessions.
7. **Feature Extraction and Leakage Check**
   - An automated leakage filter explicitly separates 24 SAFE features (used for ML) from 28 HEURISTIC features (excluded from ML to prevent proxy and direct leakage).
8. **Machine Learning Analysis**
   - XGBoost and Random Forest models process the leak-free SAFE features to classify tunneling, automation profiles, and 11 multi-label attack categories.
9. **Data Export:** Final intelligence and evaluation metrics are exported as .parquet and .json datasets for offline Jupyter Notebook analysis.

4.2 Logical Data Model — SQLite

The SQLite database acts as the central persistent storage for all ingested logs. Because logs are normalized at the edge, there are no separate raw tables for Cowrie, Honeytrap, or Suricata. Instead, the database maintains two unified tables.

4.2.1 Raw Log Tables

Field	Description
timestamp	Exact ISO 8601 time of the interaction
log_source	Origin of the log (cowrie / honeytrap / suricata)
event_type	Normalized event category (e.g., ssh_command, http_request)
src_ip / src_port	Attacker IP address and source port
dest_ip / dest_port	Target IP address and destination port
session_id	Unique session identifier from the honeypot (if applicable)
username / password	Captured credential attempts (primarily Cowrie/FTP)
request_data	Parsed request/command string
dns_query	Extracted DNS resource record name
http_metadata	HTTP method, URI, and User-Agent
raw	The original JSON event preserved as a string

4.2.2 LLM_Log Table

Field	Description
timestamp	Time of the LLM generation
session_id	Honeypot session triggering the LLM
attacker_input	The unrecognized command entered by the attacker
synthetic_response	The sanitized terminal output generated by the LLM
generation_model	Model identifier (e.g., phi3-cowrie-lora-adapter)

4.3 Logical Data Model — Offline Parquet Datasets

Real-time time-series databases (InfluxDB) have been replaced by robust columnar datasets suitable for offline Data Science workflows.

4.3.1 Session Intelligence Dataset (session_intelligence.parquet)

Contains the aggregated behavioral profiles of attackers.

Category	Example Fields	Description
Identifiers	session_id, src_ip	Unique session grouping identifiers.
SAFE Features	cmd_count, payload_entropy_max, min_inter_cmd_delay	24 strictly behavioral features used as independent variables for ML training.

HEURISTIC Features	has_download_execute, automation_score, unique_dest_ports	28 rule-based or leaking features excluded from ML models.
Target Labels	bot_label, tunnel_label, label_bruteforce, label_lateral_movement	Deterministic ground-truth labels for ML evaluation.

4.3.2 ML Evaluation Metrics (model_results.json)

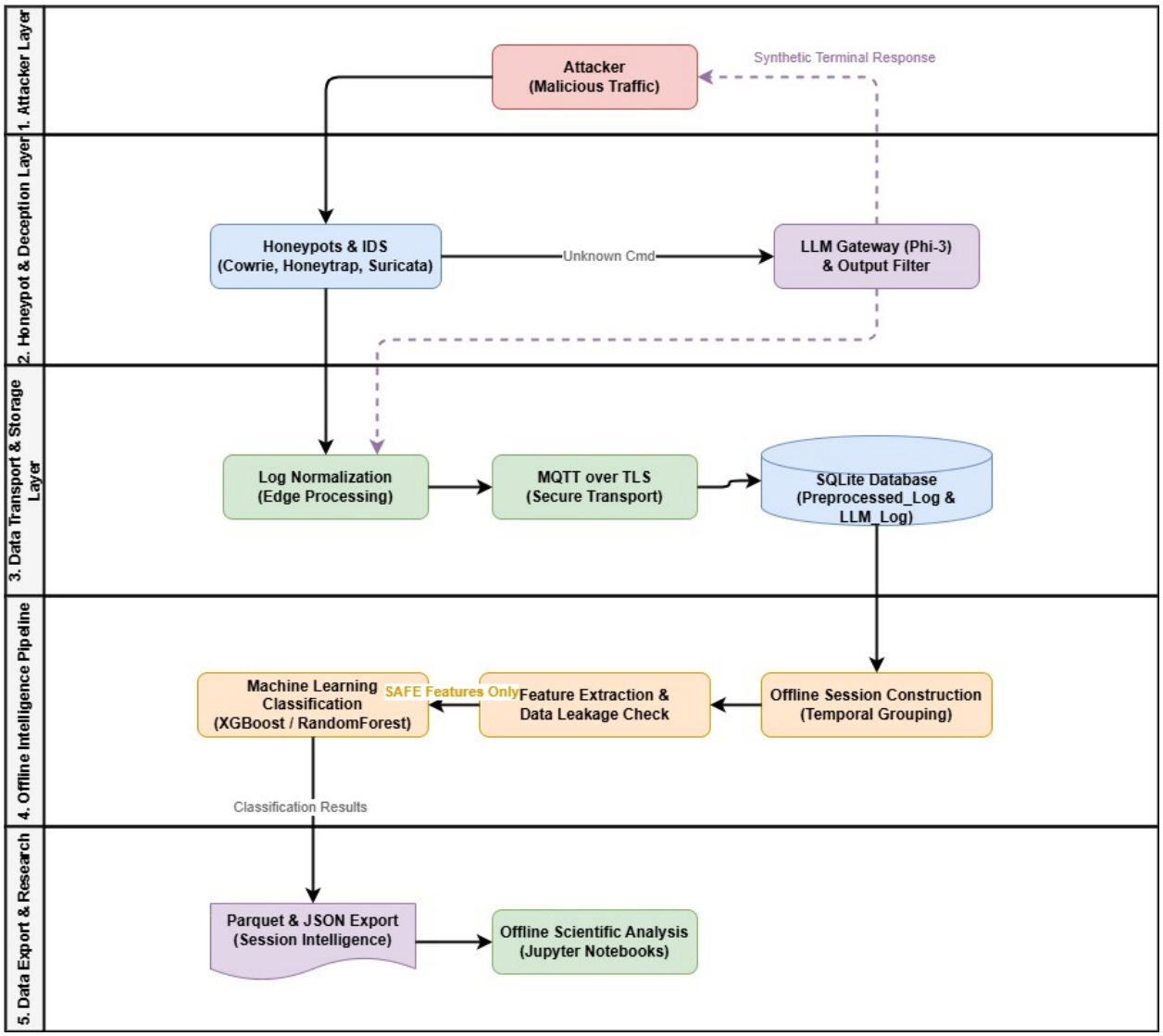
Stores the 5-Fold Stratified Cross-Validation results for model comparison.

Field	Description
model_name	XGBoost / Random Forest / Isolation Forest
target_label	The classification task (e.g., bot_label, malware_dropper)
macro_f1	Primary evaluation metric (replacing simple accuracy)
pr_auc	Precision-Recall Area Under Curve
roc_auc	Receiver Operating Characteristic Area Under Curve

4.4 Data Flow Diagram (DFD)

his section references the detailed DFD provided in the Architectural Diagrams section. The logical flow is strictly defined as:

- Attackers ↔ Honeypots & LLM Gateway (Interactive Deception Loop)
- Honeypots → Log Normalization (Edge processing)
- Normalization → MQTT → SQLite Database (Secure transport & storage)
- Database → Offline Session Construction (Temporal grouping)
- Session Construction → Feature Extraction & Leakage Check (Data safeguarding)
- SAFE Features → Machine Learning Classification (Behavioral inference)
- Classification Results → Parquet Export → Offline Analysis



4.5 Data Dictionary (High-Level)

Below is the definition of key data items and features used across the offline intelligence and LLM deception pipelines.

Log Normalization Fields (Edge to Central)

Field	Meaning
event_type	Unified category of the interaction (e.g., ssh_command, http_request)
session_id	Deterministic identifier grouping events by (src_ip, dest_ip) and time
request_data	The raw command, URI, or payload sent by the attacker

SAFE Behavioral Features (ML Independent Variables)

These 24 features are strictly used for ML training to prevent data leakage

Field	Meaning
cmd_count	Total volume of commands issued in a session
cmd_entropy	Shannon entropy of the command distribution
min_inter_cmd_delay	Minimum time gap between commands (detects automated bursts)
payload_entropy_max	Peak entropy of payloads (detects encrypted tunnels/encoded malware)
port_range_span	Numerical distance between min and max targeted ports

HEURISTIC / EXCLUDED Features (Rule-Based & Leaked)

These 28 features are explicitly excluded from ML models to ensure scientific validity

Field	Meaning
automation_score	A formulaic score (0.0 - 1.0) determining bot likelihood
credential_attempts	Raw count of login attempts (excluded due to proxy leakage)
has_download_execute	Boolean flag triggered by specific regex (e.g., wget + chmod)

LLM Deception Fields

Field	Meaning
attacker_input	The unrecognized command that triggered the LLM Gateway
synthetic_response	The sanitized, AI-generated Debian terminal output returned to the attacker
generation_model	The specific model version used (e.g., phi3-cowrie-lora-adapter)

Classification Targets (Dependent Variables)

Field	Meaning
bot_label	Binary classification (1: Bot/Scripted, 0: Manual/Human)
tunnel_label	Binary classification indicating protocol tunneling
label_<attack-type>	11 multi-label categories (e.g., label_bruteforce, label_lateral_movement)

5. Component Design

This section describes the internal components of each layer in the system architecture. Each component directly corresponds to the functional requirements defined in the SRS and reflects the offline analytical intelligence and LLM integration paradigms. The aim of this section is to provide a detailed, application-independent view of how each subsystem operates and interacts with the others.

5.1 Honeypot Layer Components

The Honeypot Layer runs on the Raspberry Pi edge device and is responsible for capturing real attacker traffic and initiating deceptive AI responses.

5.1.1 Cowrie Component (SSH Honeypot)

Responsibilities:

- Simulate SSH/Telnet services.
- Capture attacker commands, login attempts, and interactive sessions.
- Generate timestamped logs.

Inputs:

- External SSH traffic from attackers.

Outputs:

- Command logs, session logs, authentication logs.

Interactions:

- Sends log entries to the Log Aggregation Component.

5.1.2 Honeytrap Component (HTTP/FTP Emulation)

Responsibilities:

- Emulate HTTP and FTP services.
- Record HTTP requests, attempted file downloads, and scanning activity.

Inputs:

- External HTTP/FTP requests.

Outputs:

- HTTP/FTP request logs, request metadata, URIs, headers.

Interactions:

- Sends log entries to the Log Aggregation Component.

5.1.3 Suricata Component (Network IDS)

Responsibilities:

- Monitor all network packets entering the Raspberry Pi.
- Detect suspicious traffic patterns and known attack signatures.

- Produce alert logs.

Inputs:

- Raw network packets (DNS, HTTP, TCP/UDP).

Outputs:

- IDS alerts, DNS logs, HTTP metadata.

Interactions:

- Sends logs to the Log Aggregation Component.

5.2 LLM Deception Pipeline Components (Integration Layer)

This newly introduced subsystem dynamically generates context-aware synthetic responses to prevent advanced attackers from fingerprinting the honeypot.

5.2.1 LLM Gateway Component (Phi-3)

Responsibilities:

- Receive unrecognized commands from Cowrie.
- Generate contextually appropriate Debian terminal outputs using a fine-tuned causal language model (microsoft/Phi-3-mini-4k-instruct via QLoRA adapter).

Inputs:

- Unrecognized SSH commands.

Outputs:

- Raw synthetic terminal responses.

Interactions:

- Receives requests from Cowrie and passes generated responses to the Output Filter.

5.2.2 Output Filter Component

Responsibilities:

- Sanitize LLM-generated outputs to prevent prompt injection or hallucination of sensitive real-world system data.
- Ensure the generated response fits within timing and size constraints to maintain realism.

Inputs:

- Raw synthetic responses from the LLM Gateway.

Outputs:

- Safe, sanitized deceptive responses.

Interactions:

- Returns sanitized outputs to Cowrie and logs the interaction details.

5.3 Data Transfer and Storage Components

This layer ensures continuous, standardized log ingestion and structured storage for retrospective offline analysis.

5.3.1 Log Normalization & Aggregation Component (Edge)

Responsibilities:

- Read raw JSON events produced by Cowrie, Honeytrap, and Suricata using dedicated reader threads.
- Transform disparate log formats into a unified 18-field schema.
- Push normalized events into a thread-safe FIFO pipe for transmission.

Inputs:

- Raw logs from the Honeypot Layer.

Outputs:

- Normalized 18-field log entries.

Interactions:

- Feeds data into the MQTT Publisher Component.

5.3.2 MQTT Publisher Component (Edge Device)

Responsibilities:

- Publish log entries to MQTT topics.
- Maintain continuous delivery of logs.
- Retry transmission during temporary connectivity issues (as required by SRS).

Inputs:

- Standardized log entries.

Outputs:

- MQTT messages sent to the central server.

Interactions:

- Communicates with the MQTT Broker Component.

5.3.3 MQTT Broker Component

Responsibilities:

- Receive MQTT messages from the Raspberry Pi.
- Forward them to the SQLite Storage Component.
- Maintain topic subscriptions for log ingestion.

Inputs:

- MQTT messages.

Outputs:

- Log records ready for storage.

Interactions:

- Sends messages to SQLite Storage Component.

5.3.4 SQLite Storage Component

Responsibilities:

- Parse MQTT-delivered logs.
- Validate formatting and required fields.
- Store logs in structured tables.
- Store extracted features for analysis.

Inputs:

- MQTT log entries.

Outputs:

- Structured log and feature tables.

Interactions:

- Provides data to preprocessing and analysis components.

5.4 Offline Analytical Intelligence Components

This subsystem replaces real-time streaming logic. It builds behavioral profiles and trains ML models using strict scientific methodologies.

5.4.1 Session Construction Component

Responsibilities:

- Extract raw events from SQLite and group them by (src_ip, dest_ip) pairs.
- Segment event groups into temporally coherent sessions using a 30-minute inactivity gap threshold.
- Assign deterministic Session IDs to each behavioral sequence.

Inputs:

- Preprocessed logs.

Outputs:

- Aggregated session datasets containing event lists, command lists, and temporal boundaries.

Interactions:

- Provides aggregated data to the Feature Extraction Component.

5.4.2 Feature Extraction & Data Leakage Check Component

Responsibilities:

- Extract 52 raw behavioral and heuristic features across 5 categories (Command, Timing, Network, Credential, Payload/Tunneling).
- Compute automation_score and assign 11 multi-label attack categories using deterministic rules.
- Execute an automated Data Leakage Check to strictly separate 24 SAFE behavioral features from 28 HEURISTIC or label-leaking features.

Inputs:

- Constructed sessions.

Outputs:

- Leak-free SAFE feature vectors (independent variables) and ground-truth labels (dependent variables).

Interactions: Supplies only the SAFE features to the ML Classification Component.

5.4.3 ML Classification & Anomaly Detection Component

Responsibilities:

- Train tree-based models (XGBoost, Random Forest) for bot detection, tunneling detection, and 11 multi-label attack categories using only SAFE features.
- Apply Isolation Forest for anomaly detection on extremely rare classes (e.g., data exfiltration).
- Perform 5-Fold Stratified Cross-Validation to evaluate model stability.
- Compute advanced evaluation metrics (Macro-F1, PR-AUC, ROC-AUC, Feature Importance).

Inputs:

- SAFE feature vectors and standardized labels.

Outputs:

- Trained ML models, classification predictions, and metric evaluation reports.

Interactions:

- Sends results to the Data Export Component.

5.5 Data Export and Research Components

This layer replaces traditional real-time dashboards with scientific data export tools for offline research.

5.5.1 Parquet / JSON Exporter Component

Responsibilities:

- Export the final session intelligence, leak-free feature sets, and ML classification predictions as highly compressed .parquet files.
- Export cross-validation performance metrics as .json reports.

Inputs:

- ML predictions, extracted features, and metrics.

Outputs:

- Structured .parquet and .json datasets.

Interactions:

- Provides the foundational data for Offline Analysis Environments.

5.5.2 Offline Analysis Environment (Jupyter Notebooks)

Responsibilities:

- Serve as the primary UI for researchers to ingest Parquet files via Pandas DataFrames.
- Facilitate the visualization of feature importance, Macro-F1 scores, and tunneling entropy distributions through programmatic plotting (e.g., Matplotlib/Seaborn).

Inputs: Exported .parquet datasets.

Outputs: Scientific charts, summary tables, and research insights.

6. Algorithm Design

This section explains the core algorithms used in the system for constructing behavioral profiles, ensuring scientific validity via data leakage prevention, classifying cyber attacks, and generating deceptive LLM responses. All algorithms reflect the offline analytical intelligence and AI-integration paradigms established for this project.

6.1 Offline Session Construction Algorithm

This algorithm transitions the system from real-time packet inspection to robust behavioral profiling by grouping discrete raw events into temporally coherent attacker sessions.

Purpose: To aggregate millions of raw events into structured sessions based on attacker origin, target, and temporal continuity.

Input: Normalized events from the SQLite Preprocessed_Log table.

Output: An array of structured sessions, each with a unique session_id.

Algorithm Steps:

1. Retrieve raw events and sort them chronologically by timestamp.
2. Group events by unique (src_ip, dest_ip) tuple pairs.
3. Iterate through events within each group.
4. Calculate the time difference (gap) between the current event and the previous event.
5. If the gap exceeds 30 minutes: Close the current session and initialize a new session_id.
6. Aggregate all payloads, commands, URIs, and DNS queries within the closed session into pipe-delimited strings.

6.2 Data Leakage Prevention & Feature Selection Algorithm

This algorithm extracts measurable traffic features and rigorously filters them to prevent the ML models from memorizing deterministic rules (Data Leakage).

Purpose: To explicitly separate behavioral features (SAFE) from rule-based/leaked features (HEURISTIC) prior to model training.

Input: Structured sessions from Algorithm 6.1.

Output: A leak-free ML feature matrix (24 SAFE features) and derived ground-truth labels.

Algorithm Steps:

1. Compute Behavioral Features: Calculate 24 SAFE metrics per session, such as cmd_count, cmd_entropy, payload_entropy_max, and min_inter_cmd_delay.
2. Compute Heuristic Indicators: Calculate 28 deterministic metrics, including regex-based flags (e.g., has_reverse_shell) and direct threshold counters (e.g., automation_score, credential_attempts, unique_dest_ports).
3. Assign Ground-Truth Labels: Use the Heuristic Indicators to deterministically assign the 11 multi-label attack categories and the bot_label.
4. Execute Leakage Filter: Drop all 28 HEURISTIC features from the independent variable matrix.
5. Return the strictly separated SAFE feature matrix to the ML pipeline.

6.3 Machine Learning Classification Workflow

This algorithm identifies cyber attack types, protocol tunneling, and automation profiles using tree-based classifiers on the leak-free datasets.

Purpose: To classify attacker behavior using supervised (XGBoost, Random Forest) and anomaly-detection (Isolation Forest) models.

Input: SAFE feature matrix and target labels.

Output: Model predictions and performance evaluation metrics (Macro-F1, PR-AUC).

Algorithm Steps:

1. Clean numerical values and apply log-transformations to skewed features (e.g., duration_sec to log_duration).
2. Fit a StandardScaler to normalize the dataset.
3. Initialize a 5-Fold Stratified Cross-Validation loop to ensure metric stability.
4. For each fold:
    - Train XGBoost/Random Forest models targeting bot_label, tunnel_label, and 11 attack_type categories using dynamically computed scale_pos_weight for class imbalance.
    - For rare classes (e.g., data exfiltration, < 50 positive samples), train an Isolation Forest model.
5. Evaluate fold predictions strictly using the Macro-F1 score (rejecting simple accuracy).
6. Aggregate metrics and export results as .parquet and .json datasets.

6.4 LLM Command Routing Algorithm (Deception Pipeline)

This algorithm dynamically extends honeypot realism by seamlessly routing unknown attacker inputs to an AI model.

Purpose: To generate contextually appropriate, synthetic Linux terminal outputs for commands that fall outside the honeypot's static vocabulary.

Input: Raw SSH terminal input from the attacker.

Output: Synthetically generated terminal string.

Algorithm Steps:

1. Cowrie intercepts the attacker's terminal input.
2. Check if the input exists in Cowrie's static command dictionary.
3. If Yes: Execute Cowrie's default static handler and return the result.
4. If No: Forward the unrecognized command to the LLM Gateway (fine-tuned Phi-3 via QLoRA adapter).
    - Provide the LLM with the context-setting system prompt (Debian OS environment).
    - Receive the generated synthetic response.
5. Pass the generated response through the Output Filter for safety sanitization (e.g., removing hallucinated IP disclosures or toxic content).
6. Return the sanitized synthetic response to the attacker and log the exchange in the LLM_Log table.

6.5 Algorithm Dependencies

The logical flow of data through the system follows a strict dependency chain:

- **Deception Dependency:** Attacker Input → Command Routing Check → LLM Gateway Inference → Output Filter → Terminal Display.
- **Data Processing Dependency:** Edge Normalization → SQLite Storage → Offline Session Construction.
- **Intelligence Dependency:** Session Construction → Feature Extraction → Data Leakage Filter → ML Classification Workflow → Parquet Export.
- Each algorithm rigorously prepares and sanitizes the data required by the subsequent stage, ensuring no future information or rule-based proxies leak into the predictive ML models.

7. User Interface Design

This section describes the user interface components of the system, it provides two distinct types of interfaces for its two primary actors: an Analytical Interface for security researchers, and a Deceptive Terminal Interface for attackers.

7.1 Interface Overview

The system design shifts away from live monitoring toward rigorous offline intelligence and active deception.

- **Analytical Interface (For Researchers):** A data science environment (e.g., Jupyter Notebooks) used to interact with exported .parquet and .json datasets. It is designed to evaluate Machine Learning models, visualize feature importance, and analyze behavioral metrics like Macro-F1 and PR-AUC.
- **Deceptive Interface (For Attackers):** A synthetic command-line interface presented to attackers connecting to the Cowrie honeypot. For unrecognized commands, the LLM Gateway generates a realistic, context-aware Debian Linux terminal output, maintaining the illusion of a compromised system.

7.2 Analytical Interface (Jupyter & Pandas)

The visualization layer for researchers consists of programmatic data views and generated plots based on the offline session intelligence datasets.

Data Exploration Views:

- Pandas DataFrame representations of structured sessions.
- Clear tabular separation of SAFE behavioral features vs. HEURISTIC/EXCLUDED features.

Machine Learning Evaluation Plots:

- Feature Importance Bar Charts (e.g., displaying cmd_count or payload_entropy_max dominance).
- Confusion Matrices for the 11 multi-label attack categories.
- Precision-Recall Curves (PR-AUC) and ROC Curves for highly imbalanced classes (like tunneling or data exfiltration).

Cross-Validation Reports:

- JSON-formatted reports detailing the mean and standard deviation of Macro-F1 scores across the 5-Fold Stratified splits.

7.3 Deceptive Terminal Interface (LLM-Powered)

The "user interface" for the attacker is the SSH terminal prompt.

- **Cowrie Default View:** Standard Linux prompts (e.g., root@hostname:~#) and predefined static responses for known commands.
- **Dynamic LLM View:** When an attacker attempts an advanced or novel command not supported by Cowrie, the system intercepts it and presents the sanitized output generated by the Phi-3 LLM.

This ensures the attacker experiences a seamless, unbroken terminal interaction, preventing honeypot fingerprinting.

7.4 Interaction Scenarios

The system supports distinct interaction workflows based on the actor:

Scenario 1 — Researcher Evaluating ML Performance

1. The researcher loads the model_results.json and session_intelligence.parquet files into a Jupyter Notebook.
2. Executes a plotting function to visualize the 5-Fold Cross-Validation Macro-F1 scores for XGBoost vs. Random Forest.
3. Analyzes the Feature Importance chart to confirm that no excluded HEURISTIC features (e.g., automation_score) leaked into the model.

Scenario 2 — Researcher Investigating Rare Attacks

1. The researcher filters the Parquet dataset for sessions labeled label_data_exfiltration.
2. Examines the anomaly scores generated by the Isolation Forest model.
3. Investigates the raw request_data string to understand the specific commands used by the attacker.

Scenario 3 — Attacker Interacting with the Deceptive LLM

1. An attacker connects to the Cowrie honeypot via SSH and executes a complex, chained Linux command not programmed into the static honeypot.
2. The Command Router intercepts the command and queries the fine-tuned LLM Gateway.
3. The attacker receives a synthetically generated, highly realistic error or execution output.
4. Believing the system is a real, vulnerable server, the attacker continues their session, generating richer behavioral data for the ML pipeline.

8. Non-Functional Design

This section explains how the system design fulfills the non-functional requirements defined in the SRS. Each subsection corresponds directly to an NFR category and describes the design strategies used to ensure compliance in the context of offline machine learning and LLM deception.

8.1 Performance Design

The system is designed to meet strict performance expectations for offline behavioral classification and real-time LLM inference.

Design Mechanisms:

- **Offline Batch Processing Efficiency:** The session construction and feature extraction pipelines are optimized to process large datasets (e.g., aggregating 1.5 million raw events into over 60,000 sessions) efficiently using Pandas.
- **LLM Latency Optimization:** The LLM Gateway is designed to generate synthetic terminal responses with minimal latency. Fast inference is critical to prevent attackers from recognizing the delay and terminating the session prematurely.
- **Advanced Evaluation Metrics:** Traditional accuracy has been replaced; the system design is optimized to achieve a Macro-F1 score of ≥ 90% across highly imbalanced datasets using dynamically weighted tree-based models (XGBoost).
- **Lightweight Data Transfer (MQTT):** MQTT over TLS ensures efficient, secure transmission of normalized logs from the edge device to the central server.

SRS Links:

Supports NFRs: **PR-1, PR-2, PR-3, PR-4, PR-5**

8.2 Reliability Design

The system must reliably collect real attack traffic, maintain robust datasets, and handle AI integration gracefully.

Design Mechanisms:

- **Continuous Honeypot Operation:** Cowrie, Honeytrap, and Suricata run persistently on the Raspberry Pi.
- **MQTT Retry Behavior:** The publisher resends logs in case of temporary network interruptions (as required by the SRS).
- **Structured SQLite Storage:** Data is validated before insertion to prevent corruption.

SRS Links:

Supports NFRs: **RL-1, RL-2, RL-3, RL-4**

8.3 Security Design

The project intentionally exposes the Raspberry Pi to the Internet and integrates an AI model; therefore, the architecture ensures strict network isolation and LLM safety.

Design Mechanisms:

- **DMZ + Guest Network Isolation:** Raspberry Pi is placed outside internal university networks.

- **LLM Prompt Injection Mitigation:** The Output Filter sanitizes all LLM-generated responses to prevent attackers from manipulating the AI to disclose system prompts, generate toxic outputs, or hallucinate real-world sensitive data.
- **No Attack Forwarding:** Honeypots operate strictly as decoys and do not act as pivot points.

SRS Links:

Supports NFRs: **SEC-1, SEC-2, SEC-3, SEC-4, SEC-5 , SEC-6**

8.4 Usability Design

The system interfaces are designed for security researchers and data scientists who require clear, structured, and scientifically valid datasets.

Design Mechanisms:

- **Data Science Compatibility:** Exporting outputs as .parquet and .json ensures seamless native integration with Jupyter Notebooks and Pandas DataFrames.
- **Explicit Feature Separation:** The offline feature extraction pipeline clearly labels and separates SAFE features from HEURISTIC features. This design prevents researchers from accidentally introducing data leakage during future ML experiments.
- **Comprehensive Metric Reports:** Cross-validation evaluation reports automatically generate human-readable structures for Macro-F1, PR-AUC, and ROC-AUC metrics.

SRS Links:

Supports NFRs: **US-1, US-2, US-3**

8.5 Maintainability Design

The system is configured to allow updates, ML pipeline improvements, and debugging with minimal disruption.

Design Mechanisms:

- **Modular Architecture:** Each layer can be updated independently.
- **Structured Python Pipelines:** Preprocessing, Data Leakage Checking, and ML modules are organized sequentially for easy modification and automated testing.
- **Error Logging:** The system logs failures (e.g., MQTT transmission errors, corrupted entries).
- **Open-Source Publication:** The code and dataset will be published on GitHub for documentation and reproducibility purposes.

SRS Links:

Supports NFRs: **MT-1, MT-2, MT-3, MT-4**

8.6 Extensibility Design

The architecture supports adding future honeypots, advanced analysis methods, and newer LLM models.

Design Mechanisms:

- **Model-Agnostic Command Routing:** The LLM Gateway integration is designed to easily replace the current Phi-3 model with newer or more efficient language models in the future.
- **Scalable Feature Engineering:** New behavioral metrics can be added to the SAFE feature set without breaking the automated Leakage Check logic.
- **Modular ML Design:** New classifiers (e.g., semi-supervised models) can replace existing XGBoost or Isolation Forest algorithms without altering the underlying Parquet datasets.

SRS Links:

Supports NFRs: **EX-1, EX-2, EX-3, EX-4**

8.7 Portability Design

The design ensures that the system can be used in various hardware and computational environments without major modifications.

Design Mechanisms:

- **Cross-Platform Python Environment:** Analysis pipeline runs on any Linux-based central server.
- **Cloud GPU Compatibility:** The LLM fine-tuning and inference design supports execution on accessible cloud environments like Google Colab T4 instances using QLoRA.
- **Reproducible Honeypot Setup:**
  Raspberry Pi environment can be recreated on similar IoT hardware.

SRS Links:

Supports NFRs: **PT-1, PT-2, PT-3**

9. Deployment & Runtime View

This section provides the physical deployment structure of the system, along with detailed runtime scenarios that describe how components interact during operation. The deployment view reflects the real-world setup of the Raspberry Pi edge honeypot, the isolated LLM inference environment, the cloud-based MQTT broker, and the offline central analysis machine.

9.1 Physical Deployment Diagram

The system consists of highly decoupled physical and cloud environments to ensure security, performance, and strict network isolation:

1. Edge Environment (Raspberry Pi)

Running:

- Cowrie (SSH honeypot)
- Honeytrap (HTTP/FTP emulation)
- Suricata IDS
- Log aggregation service
- MQTT Publisher

Network placement:

- Exposed to the Internet via **DMZ / Guest Network**
- Isolated from internal networks (per SRS security requirements)

2. Sandboxed AI Environment (Cloud GPU / Local GPU)

Running:

- LLM Gateway (Hosting microsoft/Phi-3-mini-4k-instruct via QLoRA adapter)
- Output Sanitization Filter

Network placement:

- Operates in an isolated container (e.g., Google Colab T4 instance or air-gapped local GPU).
- Receives API requests from Cowrie but possesses no outbound network access to prevent lateral movement if compromised via prompt injection.

3. Central Analysis Environment

Running:

- SQLite database
- Python analysis scripts (Session Builder, Leakage Checker, XGBoost ML Models)
- Parquet Datasets storage
- Jupyter Notebooks (Analytical Interface)

Network placement:

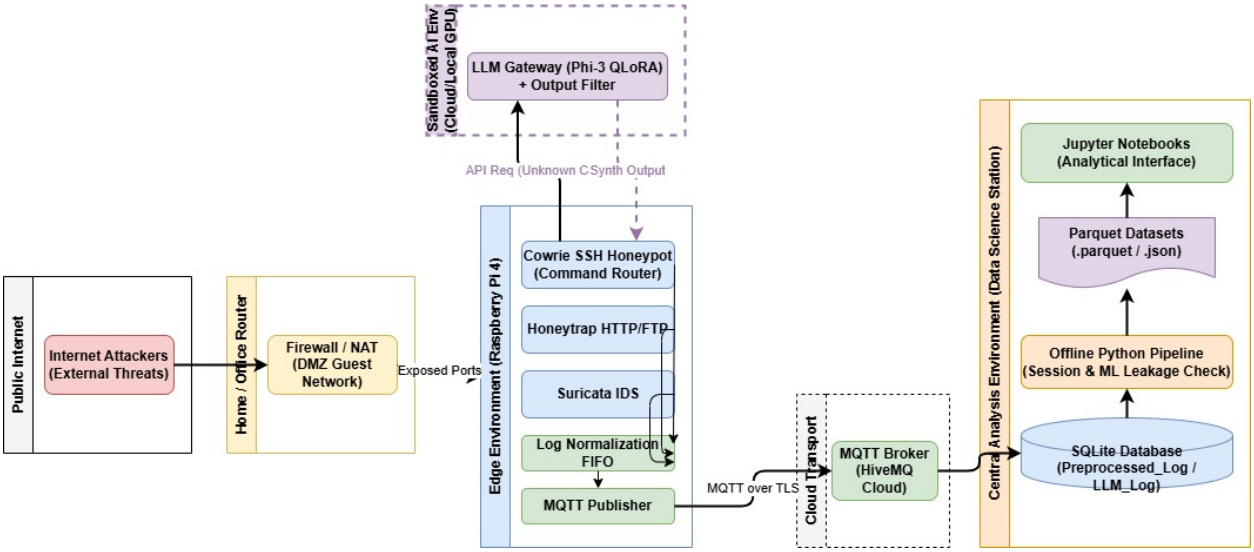
- Local secure network (not exposed to attackers). Subscribes to the Cloud MQTT Broker.

4. Cloud Transport Layer

- Cloud MQTT Broker (e.g., HiveMQ Cloud)

Network Placement:

- Facilitates secure TLS communication between the Edge Pi and the Central Analysis Machine.



9.2 Runtime Scenarios (Sequence Diagrams)

The following runtime scenarios describe the operational flow of the system during its two most critical processes: real-time attacker deception and offline behavioral intelligence generation.

9.2.1 Runtime Scenario 1 — LLM Deception & Edge Logging (Real-Time)

Purpose: To describe how the system interactively handles an attacker using an unrecognized command, deceiving them with AI while securely logging the event.

Sequence Steps:

1. Attacker sends a complex, unrecognized SSH command to the Raspberry Pi.
2. Cowrie Honeypot intercepts the command via its Command Router.
3. Cowrie queries the LLM Gateway (Phi-3) through an internal API.
4. The LLM Gateway generates a context-aware Debian terminal response and passes it through the Output Filter for sanitization.
5. The sanitized synthetic response is returned to Cowrie.
6. Cowrie displays the synthetic output to the attacker, extending the session duration.

7. Simultaneously, Cowrie forwards the log data (both the raw event and the LLM interaction) to the Log Aggregator (FIFO), which publishes it via MQTT to the central SQLite Database (Preprocessed_Log and LLM_Log).

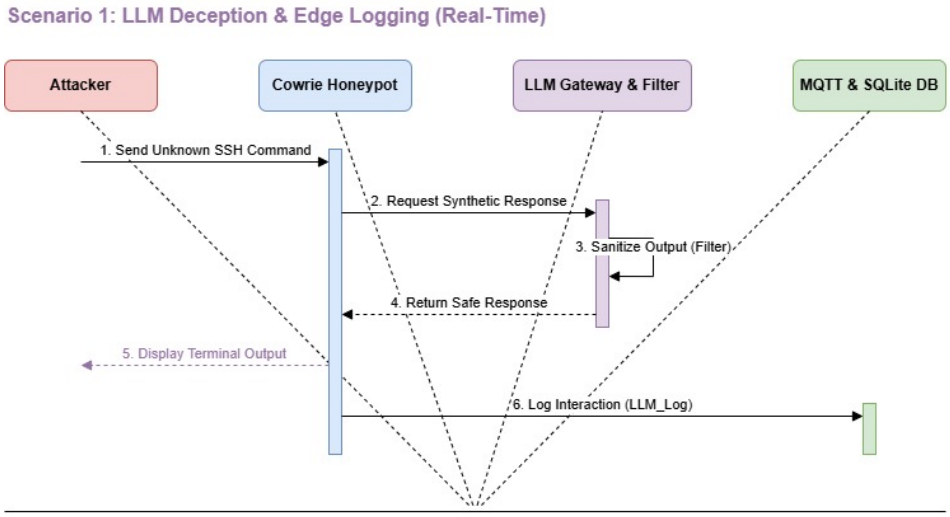
9.2.2 Runtime Scenario 2 — Offline ML Intelligence Pipeline (Batch Processing)

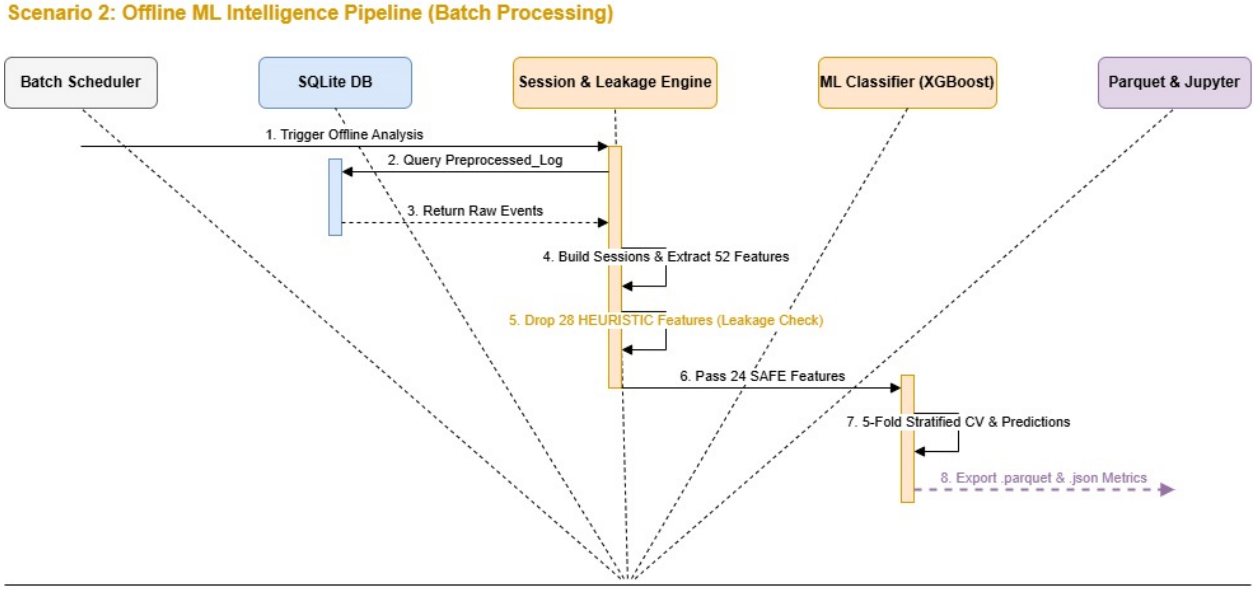
Purpose: To describe the leakage-aware, offline machine learning workflow that transforms raw logs into scientific intelligence datasets.

Sequence Steps:

1. The Batch Scheduler / Researcher triggers the offline analysis pipeline.
2. The Session Builder queries the SQLite Preprocessed_Log table to retrieve all raw events.
3. Raw events are grouped into temporally coherent sessions using a 30-minute inactivity gap.
4. The Feature Extractor computes 52 raw traffic characteristics.
5. The Data Leakage Checker explicitly drops the 28 HEURISTIC features (e.g., automation_score, credential_attempts), retaining only the 24 SAFE behavioral features.
6. The SAFE features are passed to the ML Classification Module (XGBoost / Isolation Forest).
7. The models perform 5-Fold Stratified Cross-Validation and generate predictions (Bot/Human, Tunneling, 11 Attack Categories).

8. The final session intelligence and Macro-F1 evaluation metrics are exported to the Jupyter Analysis Environment as .parquet and .json datasets.





10. SRS Traceability

This section maps the functional and non-functional requirements of the SRS to the corresponding design elements of the SDD. The goal here is to ensure that the design fully meets the requirements and that all architectural, data, algorithmic, and interface decisions (especially the transition to offline intelligence and LLM deception) are justified by the updated SRS.

10.1 Functional Requirements Traceability

Table 10.1 – Mapping SRS Functional Requirements (FR) to SDD Sections

SRS Requirement ID	SRS Requirement Description	SDD Section(s) Addressing It
FR-1.1	Run Cowrie to log SSH-based attacks and commands.	3.2.2, 5.1.1
FR-1.2	Run Honeytrap to emulate HTTP/FTP services.	3.2.2, 5.1.2
FR-1.3	Run Suricata to generate IDS alerts and metadata.	3.2.2, 5.1.3
FR-1.4	Timestamp and store all attacker interactions in raw logs.	3.2.2, 4.2.1
FR-1.5	Operate Raspberry Pi in DMZ/guest network mode.	3.2.1, 9.1
FR-2.1	Collect logs using dedicated reader threads.	5.3.1
FR-2.2	Normalize events into a unified 18-field schema.	5.3.1, 4.2.1
FR-2.3	Aggregate normalized logs via a FIFO pipe.	5.3.1
FR-3.1	Transmit logs via MQTT over TLS (Port 8883).	3.2.3, 5.3.2
FR-3.2	Ensure uninterrupted log forwarding.	5.3.2
FR-3.3	Retry transmission in case of connectivity issues.	5.3.2
FR-4.1	Store all received log data in a SQLite database.	3.2.3, 5.3.4
FR-4.2	Maintain Preprocessed_Log and LLM_Log tables.	4.2.1, 4.2.2, 5.3.4
FR-5.1	Perform offline aggregation of raw logs into sessions.	5.4.1, 6.1

FR-5.2	Group events using a 30-minute inactivity gap.	5.4.1, 6.1
FR-5.3	Assign deterministic Session IDs.	5.4.1, 6.1
FR-6.1	Extract 52 raw features per session.	5.4.2, 6.2
FR-6.2	Assign 11 multi-label attack categories via rules.	5.4.2, 6.2
FR-6.3	Compute automation score for Bot/Manual profiling.	5.4.2, 6.2
FR-6.4	Export intelligence as .parquet and .json.	5.5.1, 4.3
FR-7.1	Separate features into SAFE and HEURISTIC.	5.4.2, 6.2
FR-7.2	Execute Data Leakage checks automatically.	5.4.2, 6.2
FR-7.3	Strictly use only 24 SAFE features for ML models.	5.4.2, 6.2
FR-8.1	Train tree-based ML models (XGBoost, Random Forest).	5.4.3, 6.3
FR-8.2	Classify bot behavior, tunneling, and 11 attack types.	5.4.3, 6.3
FR-8.3	Employ Isolation Forest for rare anomaly detection.	5.4.3, 6.3
FR-8.4	Evaluate using Macro-F1 score ($\geq$ 90%).	5.4.3, 6.3, 8.1
FR-8.5	Implement 5-Fold Stratified Cross-Validation.	5.4.3, 6.3
FR-9.1	Intercept unknown commands via Cowrie Command Router.	3.2.4, 5.1.1, 6.4

FR-9.2	Generate outputs using fine-tuned Phi-3 LLM.	3.2.4, 5.2.1, 6.4
FR-9.3	Apply Output Filter to sanitize LLM responses.	5.2.2, 6.4
FR-9.4	Log synthetic AI interactions to LLM_Log table.	4.2.2, 5.3.4

10.2 Non-Functional Requirements Traceability

Table 10.2 – Mapping SRS NFRs to SDD Design Decisions

NFR ID	Requirement Description	SDD Section Addressing It
PR-1	≥ 90% Macro-F1 for Bot/Human & Tunneling.	8.1, 5.4.3, 6.3
PR-2	≥ 90% Macro-F1 for 11 ML Attack Categories.	8.1, 5.4.3, 6.3
PR-3	Efficient offline session processing via Pandas.	8.1, 6.1, 5.4.1
PR-4	SQLite must handle large log volumes natively.	8.1, 4.2
PR-5	LLM Gateway must ensure low latency inference.	8.1, 5.2.1
RL-1	Continuous honeypot edge operation.	8.2, 5.1
RL-2	Reliable MQTT delivery via TLS.	8.2, 5.3.2
RL-3	Secure log validation against database corruption.	8.2, 5.3.4
RL-4	LLM Fallback to static Cowrie on AI failure.	8.2, 6.4
SEC-1	DMZ/Guest isolation for Raspberry Pi.	8.3, 9.1

SEC-2	No connection to internal production networks.	8.3, 9.1
SEC-3	Restricted access to local SQLite/Parquet files.	8.3
SEC-4	Honeypots do not relay or forward attacks.	8.3, 5.1
SEC-5	LLM Prompt Injection mitigation (Output Filter).	8.3, 5.2.2
SEC-6	Sandboxed execution of LLM inference container.	8.3, 9.1
US-1	Export structured .parquet for Jupyter Notebooks.	8.4, 7.2
US-2	Clearly distinguish SAFE from HEURISTIC features.	8.4, 4.3.1
US-3	Automatically generate structured Metric Reports.	8.4, 4.3.2, 7.2
MT-1	Modular configurations for edge/MQTT/DB.	8.5, 5.3
MT-2	Maintainable Python analysis/ML pipelines.	8.5, 5.4
MT-3	Error logging for MQTT and LLM components.	8.5, 5.3.2
MT-4	Code and dataset published open-source on GitHub.	8.5
EX-1	Unified schema supports new Honeypot/IDS tools.	8.6, 5.3.1
EX-2	Extensible ML design for new algorithms.	8.6, 5.4.3
EX-3	Allow feature additions without breaking Leakage Check.	8.6, 5.4.2

EX-4	Model-agnostic LLM Command Routing architecture.	8.6, 5.2.1
PT-1	Environment runs on any Linux central server.	8.7, 9.1
PT-2	Platform-independent Python Data Science stack.	8.7
PT-3	Reproducible Raspberry Pi setup.	8.7, 9.1